
Study Notes: Generative AI Evolution & LLM Foundations

Part 1: LangChain, LLMs & Prompt Engineering | Sheryians AI School (Akash Vyas)

1. Course Introduction & Overview

Main Concepts

- **Course Context:** This video marks Part 1 ("LLM Foundations") of a 4-part Generative AI series hosted by Akash Vyas on *Sheryians AI School*.
- **4-Part Series Roadmap:**
 - **Part 1 — LLM Foundations:** Core concepts of Large Language Models, APIs, LangChain, prompt engineering, embeddings, and structured outputs.
 - **Part 2 — RAG Systems:** Retrieval-Augmented Generation connecting LLMs to external documents and vector stores.
 - **Part 3 — Agentic AI:** Building autonomous multi-agent AI systems using frameworks like LangGraph.
 - **Part 4 — End-to-End Capstone Project:** Developing and deploying a scalable, production-ready GenAI application.

5 Foundational Prerequisites for Generative AI

Before building Generative AI applications, five foundational pillars are recommended:

- **1. Python Programming:** The primary language used for implementing GenAI logic and managing ecosystem libraries.
- **2. Machine Learning (ML) Fundamentals:** Basic understanding of ML concepts and algorithms required for technical interviews and system design.
- **3. Deep Learning (DL) Concepts:** Neural networks, backpropagation, and model training mechanics.
- **4. Transformer Architecture:** The core neural network architecture introduced in research that powers modern GenAI models (e.g., GPT).
- **5. Natural Language Processing (NLP) Basics:** Fundamental text-processing concepts such as tokenization and word embeddings.

2. The 4-Phase AI Evolution (The GenAI Ecosystem)

Ecosystem Architecture

The GenAI Lifecycle:

[Phase 1: Research Papers] → [Phase 2: Bedrock LLMs] → [Phase 3: Providers / APIs] → [Phase 4: Developer Applications]

- **Phase 1 — Research Layer:** Academic researchers and scientists (e.g., Google Brain) publish groundbreaking research papers introducing architectural innovations (e.g., LSTM, GRU, and "*Attention Is All You Need*" introducing Transformers).

-
- **Phase 2 — Bedrock Models (Foundation Models):** Large technology organizations (OpenAI, Google, Meta, Anthropic, xAI) take published research papers and invest massive capital and GPU supercomputing clusters to train foundation models (ChatGPT/GPT-4, Gemini, Claude, Grok, LLaMA).
 - **Phase 3 — Providers:** Platforms that host bedrock models and expose access to developers via API endpoints or open-source hubs (OpenAI API Platform, Google AI Studio, Grok Cloud, Hugging Face).
 - **Phase 4 — Developer / Application Layer:** AI engineers leverage API keys or local models along with orchestration frameworks (like LangChain) to build real-world end-user applications.

3. Large Language Models (LLMs) – Core Architecture

Breakdown of 'Large Language Model'

- **Large:** Trained on billions of words across massive datasets (books, websites, Wikipedia, code repositories, conversation logs) using GPU supercomputers.
- **Language:** Capable of understanding and generating both human spoken languages (English, Hindi, Spanish) and programming languages (Python, JavaScript, SQL).
- **Model:** A deep learning neural network system that learns statistical patterns from training data.

How LLMs Work Under the Hood

- **Pattern Matching & Token Prediction:** LLMs do *not* search Google live for every prompt, nor do they "think" like a human brain.
- **Numerical Processing:** Text inputs are converted into dense numerical vector representations called **embeddings**.
- **Next-Word Prediction:** The model evaluates statistical patterns across its numerical training data and predicts the single most likely next token sequentially until the response is complete.

Model Distribution Types

- **Closed-Source / Paid Models:** Proprietary models accessed via paid API credits (e.g., OpenAI GPT-4o/GPT-5, Google Gemini Pro, Anthropic Claude).
- **Open-Source / Free Models:** Models whose weights are publicly downloadable and hostable locally or run via community hubs (e.g., Meta LLaMA, Mistral, DeepSeek, TinyLlama).

4. The LangChain Framework & Core Components

Why LangChain?

Every model provider (OpenAI, Google, Anthropic, Grok, Mistral) uses a unique SDK, syntax, and response format. Writing custom code for dozens of models makes codebases brittle. **LangChain** provides a standardized, unified interface so developers write code once and seamlessly connect to any LLM provider.

The 6 Core Components of LangChain

- **1. Models:** Connects applications to LLM providers (Chat Models for conversational text, Embedding Models for vector representations, Multimodal Models for text/audio/vision).
- **2. Prompts:** Structures instructions sent to models (Simple Prompts, System + User Prompts, Prompt Templates, Structured Prompts).
- **3. Chains (Runnables):** Sequences connecting multiple processing steps or tasks into a single pipeline.

- **4. Memory:** Persists chat history across turns, maintaining conversational context.
- **5. Indexes:** Structures external documents (PDFs, CSVs, databases) into search-optimized indices for RAG.
- **6. Agents:** Dynamic AI systems equipped with tools (web search, calculators, database querying) that decide execution steps autonomously.

5. Model Execution: API vs. Local Deployment

Technical Environment Setup

- **Virtual Environments (uv venv):** Isolated environments created per project to keep dependencies contained.
- **uv Package Manager:** A Rust-based Python package manager executing installations 10x to 100x faster than standard pip.
- **Environment Variables (.env):** API keys are safely loaded using libraries like python-dotenv to avoid hardcoding credentials.

Comparison: API-Based vs. Local Execution

Feature	API-Based Execution	Local Model Execution
How it Works	Sends HTTP requests to provider cloud endpoints via API keys.	Downloads weights to local disk and runs on local GPU/CPU.
Storage & Hardware	Negligible local storage and minimal computing power required.	Requires large disk space (TinyLlama ~2.2GB; DeepSeek ~700GB) & GPU VRAM.
Cost & Limits	Charged per token; subject to vendor rate limits.	100% free unlimited execution once downloaded.
Data Privacy	Data travels to external cloud servers.	Completely private, air-gapped, and offline-capable.

Key Model Hyperparameters

- **temperature (0.0 to 1.0):** Controls randomness. 0.0 produces deterministic, logical outputs for math/coding; 0.7–1.0 increases creativity for poetry and brainstorming.
- **max_tokens:** Imposes a hard limit on the maximum number of generated tokens to control API costs and prevent runaway responses.

6. Prompt Engineering Types & Roles

- **Simple Prompt:** Single direct instruction (e.g., *"Explain Machine Learning"*).
- **System + User Prompt (Role Separation):**
 - **System Message:** Defines overall persona, behavioral constraints, and instructions for the AI (e.g., *"You are a helpful AI teacher"*).
 - **Human/User Message:** The actual query or task provided by the user.
 - **AI Message:** The assistant's generated response stored in the conversation log.
- **Prompt Templates:** Reusable layouts containing variable placeholders (e.g., Explain {topic} in simple terms) filled dynamically at runtime using prompt.invoke(...).

7. Structured Outputs & Pydantic Validation

Why Structured Outputs Matter

Standard natural language text is suitable for human reading, but backend databases (SQL, MongoDB) and computational pipelines require strict, machine-readable formats like JSON.

Schema Definition & Parsing Workflow

1. Define Schema (Pydantic BaseModel): Define data structures, field types (str, int, List, Optional), and constraints.

2. Output Parser (PydanticOutputParser): Injects formatting instructions into the prompt and validates/parses raw JSON into structured Python objects.

```
from pydantic import BaseModel
from typing import List, Optional
from langchain_core.output_parsers import PydanticOutputParser

# Step 1: Define strict schema
class MovieData(BaseModel):
    title: str
    release_year: Optional[int]
    genres: List[str]
    director: Optional[str]
    rating: float
    summary: str

# Step 2: Initialize parser
parser = PydanticOutputParser(pydantic_object=MovieData)
```

8. Technical Glossary & Terminology

Term	Simple Definition	Context in Video	Short Example
LLM	Deep learning model trained on massive text to generate human language.	Core foundation of GenAI apps.	GPT-4o, Gemini 1.5, Claude 3
Transformers	Neural network architecture using attention mechanisms.	Theoretical basis for models processing context.	GPT ("Pre-trained Transformer")
Tokens	Text broken down into smaller numerical sub-word units.	Calculates API costs, memory, and text limits.	"Generative AI" → ["Generative", "AI"]
Embeddings	High-dimensional numerical vector representations of text.	Used in semantic search and RAG systems.	"Rocket Science" → [0.6, 0.2, 1.9, ...]
LangChain	Open-source framework unifying syntax across LLM providers.	Main library used to orchestrate GenAI components.	init_chat_model("gpt-4o")
Bedrock Model	Heavy foundation models built using GPU supercomputers.	Phase 2 of the GenAI ecosystem.	Base ChatGPT or Meta LLaMA
uv	Fast Rust-based Python package & environment manager.	Used to create isolated virtual environments quickly.	uv venv / uv pip install
System Message	Prompt setting persona, behavioral rules, and tone.	Used to enforce chatbot behavior/roles.	"You are a concise AI tutor."

Term	Simple Definition	Context in Video	Short Example
Prompt Template	Reusable prompt layout with variable placeholders.	Extracts metadata dynamically from text.	Explain {topic} simply.
Pydantic Schema	Python structure enforcing strict data types on outputs.	Guarantees clean JSON for databases.	Enforces release_year as int.
Temperature	Hyperparameter controlling randomness in generation.	Adjusted for logical vs. creative tasks.	0.0 for code, 0.8 for creative writing.

9. Quick Revision Summary

Key Takeaways for Exam / Interview Revision

- 5 Foundational Prerequisites:** Python, ML Fundamentals, Deep Learning, Transformer Architecture, and NLP Basics.
- 4-Phase AI Evolution:** Research Papers → Bedrock LLMs → Providers / APIs → Developer Applications.
- LLM Working Principle:** Models convert text to numerical embeddings and predict the next token based on statistical patterns (not live search).
- LangChain Core Purpose:** Eliminates SDK fragmentation with a single unified syntax interface for OpenAI, Google Gemini, Anthropic, Grok, Mistral, and Hugging Face.
- 6 LangChain Components:** Models, Prompts, Chains (Runnables), Memory, Indexes, and Agents.
- API vs. Local Deployment:** API execution is fast and lightweight on disk but incurs token costs; Local execution (e.g., TinyLlama) is free and private but hardware-intensive.
- Prompt Roles:** System Message (sets persona/rules), Human Message (user query), and AI Message (assistant response).
- Structured Outputs:** Use Pydantic BaseModel and PydanticOutputParser to force LLMs to return validated, machine-readable JSON.